

Computer Architecture Project 1

Member and Teamwork

B05902003 李哲安: EX/MEM/WB stages, Forwarding

B05902049 簡崇安: IF/ID stages, Hazzard Detection

B05902109 柯上優: testbench, wire connection, report

Execute

- environment: linux
- language: verilog
- require package: iverilog
- compile and run

```
cd code
bash run.sh
```

Implement pipeline CPU

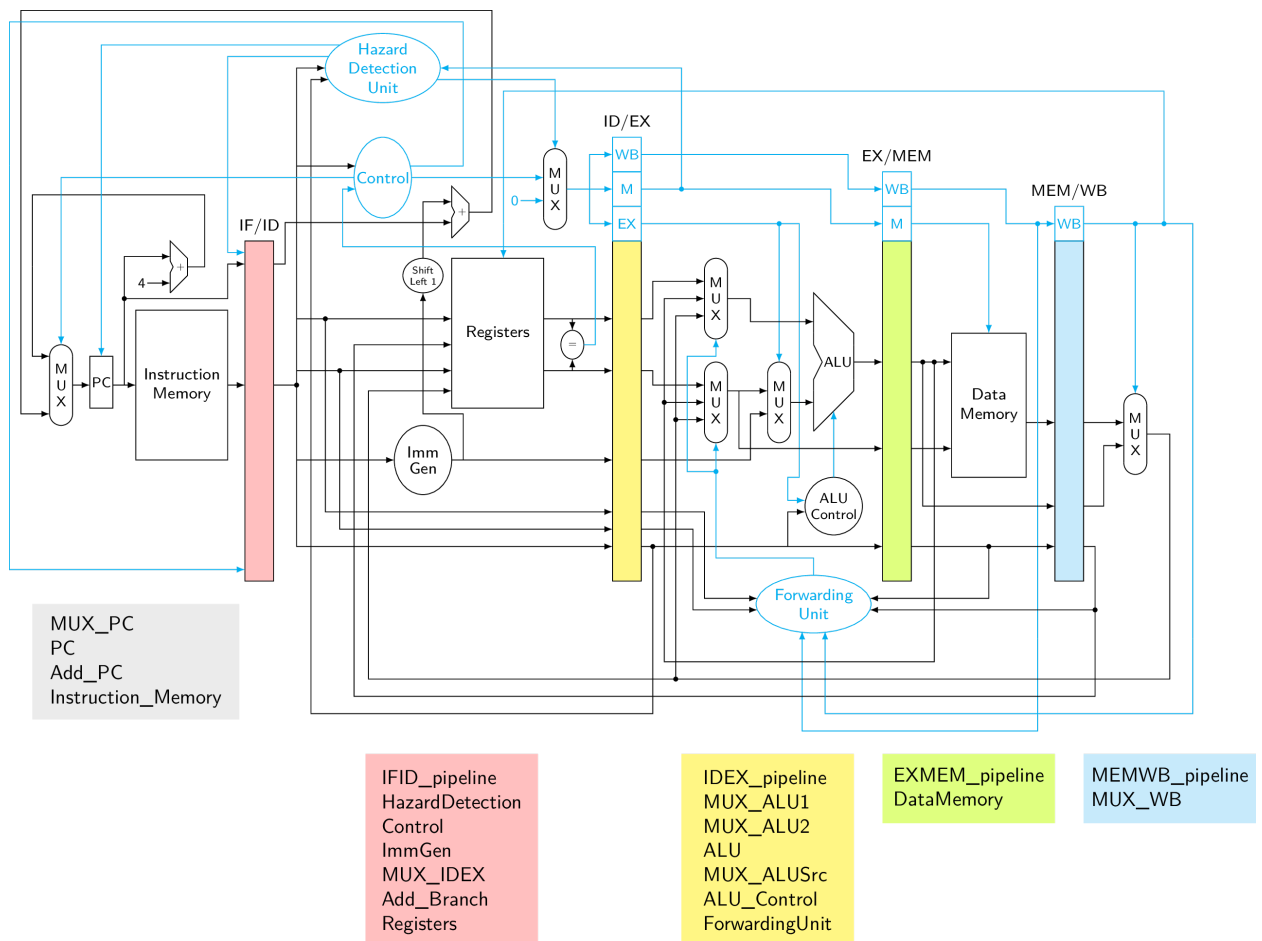
- 於報告最後面附上完整圖片
- 使用clk刺激PC與四個pipeline，posedge會讀入前一個stage傳來的資料，negedge會開始將資料流給右邊的stage。此方法讓五個部分有條理的運行，保證不讓同一個stage同時處理兩個以上的instructions。

Implement each module

- 比較值得一提，且又是上一次作業沒有處理到的module：
 - Forwarding Unit + 兩個ALU前面的MUX：
 - Forwarding功能使的instruction不必stall等待前面的資料寫回Registers。藉由判斷MEM/WB stages要寫回Registers且MEM_rd/WB_rd等於EXE_rs1/2，可以提早將尚未寫回去的資料先拿過來使用。
 - Hazzard Detection + PC前面的MUX：
 - 此次作業要求ld後面若立刻使用剛load進來的register，則需要stall到資料拿到且可以使用Forwarding提取為止。
 - 判斷的方法是對於要Memory read的instruction，假如接續的instruction就要使用到(IDEX_rd == IFID_rs1 || IDEX_rd == IFID_rs2)，就傳送信號給「PC」、「IF/ID_pipeline」、「Control剛算到的E/M/WB」，前者重新跑一次這個instruction，後兩者洗掉，實現stall。
 - 增加Jump功能的Control：
 - 此版本的結構，我們在ID stage就先判斷register[rs1] == register[rs2]。若成立，則搶先將Sign Extend的imm左移1、加上此刻的instruction address，傳回PC進行jump。此外，會flush掉提前進入IF stage的一個instruction，取消掉因為我們平行處理而多處理到的部分。

Problems and Solution

- 最一開始沒有pipeline要用posedge和negedge控制寫入寫出的概念，同一個cycle時多個instruction一次執行，造成forwarding與hazard detection嚴重堆積。
 - 由謝議霆與李澤諺同學們的指導下解決問題。
- data_memory和registers的資料存取，與範例輸出不同cycle，我們的早一個cycle。
 - 將data_memory和registers加入clk_i，只在posedge時存取，這樣就會延遲到下一個cycle才更新。
- 在新增了Fibonacci的測資後，發生了以下問題：
- jump跳到一個8000多的PC位置。
 - 由於以前沒考慮過負數的jump rd位置，所以在修正了add_Branch後解決了
- 發生了先輸出register[addr]的值，後更新register[addr]的問題。
 - 將register改成non blocking的更新方式。
- 儘管現在的程式是對的，但是在某些instruction會發生「提前將register的內容更新，以致於前一個cycle一次改變兩個register內的值，後一個cycle的輸出值疑似沒改變」的狀況，但是最終結果是對的，僅限於「連續R-type instruction」的過程
 - 尚未解決，由於助教保證提前一個cycle輸出可以接受，所以便放心地繳交了。



- 這才是正確且完整的datapath，直到report完成的時間點(2018/12/5 23:00)，作業引導裡付上的圖片都有漏接線路與缺少元件，如：
 - WB/M/EX各自的尾端都沒接完，只停在pipeline裡面。
 - 少了ALU Control Unit和ALU的input data 2前需要一個MUX來保存值。